Министерство образования Республики Беларусь
Учреждение образования
Белорусский Государственный университет
информатики и радиоэлектроники

Инженерно-экономический факультет
Кафедра экономической информатики

Методическое пособие

по дисциплине «Современные технологии серверной разработки»

на тему «Построение CI/CD-пайплайна на GitHub Actions на примере проекта TaskService»

Выполнили:

Архипец Д.А.
студент группы 472301
Писарик М.И.
студент группы 472301

Под руководством:

Петрович Н.О.
старший преподаватель
кафедры ЭИ

Минск 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4
1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ CI/CD И GITHUB ACTIONS 5
1.1 Непрерывная интеграция и непрерывная доставка 5
1.2 GitHub Actions как платформа CI/CD 6
1.3 Структура workflow-файла 6
1.4 Триггеры, права и параллельность 8
1.5 Job'ы, шаги, матрица и сервисы 8
1.6 Воспроизводимость и кэширование с uv 9
2. АНАЛИЗ ПРОЕКТА TASKSERVICE 10
2.1 Назначение и функциональность 10
2.2 Архитектура приложения 11
2.3 Структура репозитория 12
2.4 Технологический стек и требования к окружению 13
2.5 Организация тестирования 14
2.6 Предпосылки внедрения CI 15
3. ПРОЕКТИРОВАНИЕ CI-ПАЙПЛАЙНА 16
3.1 Требования к пайплайну 16
3.2 Архитектура пайплайна 16
3.3 Выбор событий и политики запуска 17
3.4 Разделение на job'ы и кэширование 17
3.5 Работа с секретами 18
4. РЕАЛИЗАЦИЯ WORKFLOW CI.YAML 18
4.1 Заголовок workflow и политика запуска 18
4.2 Job check: линтинг, форматирование и типы 19
4.3 Job tests: матрица и сервисы 20
4.4 Генерация JWT-ключей 22
4.5 Запуск тестов и сбор покрытия 22
4.6 Публикация покрытия в Codecov 23
4.7 Эволюция пайплайна и типичные ошибки 23

5. АНАЛИЗ РЕЗУЛЬТАТОВ РАБОТЫ ПАЙПЛАЙНА 25
5.1 Статистика запусков 25
5.2 Успешный запуск #32 27
5.3 Анализ сбоев 29
5.4 Покрытие и Codecov 31
5.5 Оценка эффективности и рекомендации 31
6. CD: НЕПРЕРЫВНАЯ ДОСТАВКА И РАЗВЁРТЫВАНИЕ 32
6.1 От CI к CD 32
6.2 Исторический CD-workflow проекта 32
6.3 Почему CD требует больше, чем CI 34
6.4 Рекомендуемая схема CD 34
6.5 Секреты и безопасность CD 37
7. ЛОКАЛЬНОЕ ВОСПРОИЗВЕДЕНИЕ И СОПРОВОЖДЕНИЕ ПАЙПЛАЙНА 37
7.1 Команды, совпадающие с CI 37
7.2 Pre-commit как первый барьер 38
7.3 Обновление зависимостей и action'ов 39
7.4 Чек-лист при изменении workflow 39
ЗАКЛЮЧЕНИЕ 40
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 40
ПРИЛОЖЕНИЕ А. ПОЛНЫЙ ФАЙЛ CI.YAML 42
ПРИЛОЖЕНИЕ Б. ШПАРГАЛКА ПО КОМАНДАМ 45
Б.1 Команды GitHub CLI для работы с Actions 45
Б.2 Локальные команды проекта 45
Б.3 Минимальный шаблон workflow 46

ВВЕДЕНИЕ

Современная серверная разработка невозможна без автоматизации проверки качества кода. Чем больше проект и чем чаще в него вносятся изменения, тем выше риск, что новая функция нарушит уже существующее поведение, а ошибка попадёт в основную ветку и будет обнаружена только на стенде или у пользователя. Именно поэтому в индустрии получили распространение практики непрерывной интеграции (CI) и непрерывной доставки (CD), а платформы вроде GitHub Actions стали стандартным инструментом их реализации.

В качестве примера рассматривается реальный учебный проект TaskService — REST API для управления задачами на FastAPI с аутентификацией JWT, разграничением ролей, асинхронной работой с PostgreSQL и Redis, фоновыми задачами Celery и полным стеком наблюдаемости MELT. Проект содержит 75 автотестов, набор инструментов статического анализа и готовый workflow GitHub Actions, который запускается на каждый push и pull request в ветку main.

Цель пособия — сформировать практические навыки проектирования, реализации и анализа CI/CD-пайплайна на примере GitHub Actions и реального проекта TaskService.

Для достижения цели необходимо решить следующие задачи:

1. изучить теоретические основы CI/CD и ключевые понятия GitHub Actions;
2. проанализировать структуру проекта TaskService, его тесты и требования к окружению;
3. спроектировать пайплайн: определить триггеры, состав job'ов, матрицу версий Python и сервисы;
4. разобрать реализацию файла .github/workflows/ci.yaml по шагам;
5. проанализировать результаты реальных запусков, покрытие и типичные сбои;
6. рассмотреть переход от CI к CD и требования к безопасному развёртыванию.

Объект исследования — процесс автоматизации проверки качества серверного приложения. Предмет исследования — CI/CD-пайплайн на базе GitHub Actions. Пособие построено по принципу «от теории к практике»: сначала вводятся понятия и синтаксис, затем они закрепляются на конкретном проекте, а завершается работа анализом реальных результатов и рекомендациями.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ CI/CD И GITHUB ACTIONS

1.1 Непрерывная интеграция и непрерывная доставка

Continuous Integration (CI) — это практика, при которой разработчики регулярно объединяют свои изменения в общую ветку, а каждое объединение автоматически проверяется: выполняется сборка, линтинг, форматирование, проверка типов и тесты. Главная идея CI — обнаружить проблему как можно раньше, пока стоимость её исправления минимальна.

Continuous Delivery (CD) — продолжение CI: после успешных проверок приложение автоматически готовится к выпуску — собирается артефакт (например, Docker-образ), публикуется в реестре и доставляется в целевую среду. Если развёртывание выполняется автоматически, без ручного подтверждения, говорят о Continuous Deployment. На практике термин CD часто используют обобщённо для обеих практик.

Совокупность автоматизированных этапов от коммита до работающего приложения называют пайплайном. Типовая схема CI/CD приведена на рисунке 1.

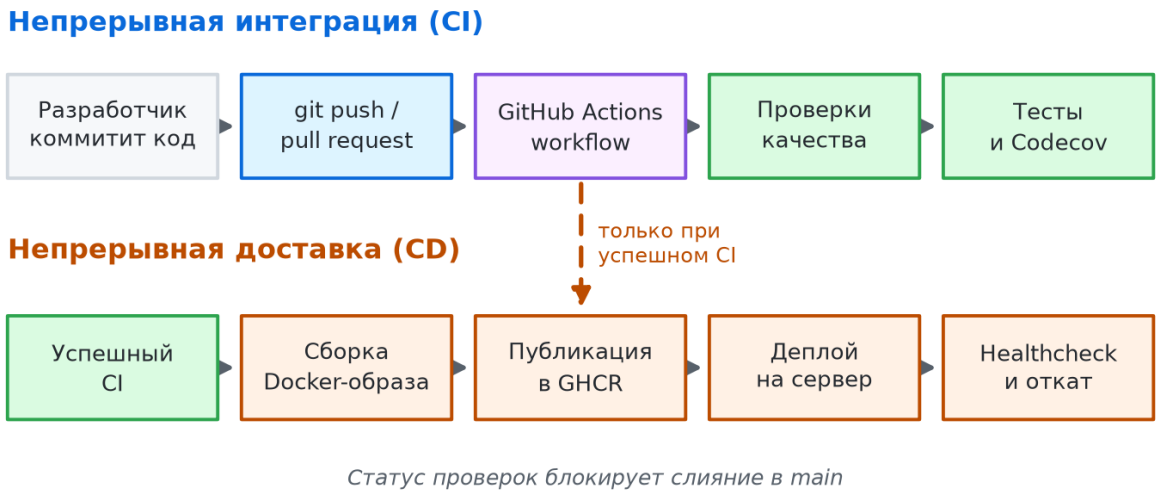


Рисунок 1 – Общая схема CI/CD-пайплайна

CI/CD даёт следующие эффекты:

- раннее обнаружение ошибок — проверки выполняются до слияния в основную ветку;
- воспроизводимость — все участники и CI используют одинаковые версии Python и зависимостей;
- экономия времени — рутинные операции (линтинг, тесты, отчёты) выполняются автоматически;

- прозрачность — статус проверок виден в pull request, а история запусков хранится в GitHub;
- готовность к поставке — успешный CI является предпосылкой для автоматического CD.

1.2 GitHub Actions как платформа CI/CD

GitHub Actions — платформа для реализации CI/CD-пайплайнов, встроенная в GitHub. Она позволяет автоматизировать повторяющиеся задачи: тесты, линтинг, форматирование, проверку типов, сборку и деплой. Workflow описываются YAML-файлами и хранятся в репозитории рядом с кодом, поэтому изменения в пайплайне проходят такое же ревью, как и изменения в приложении.

Ключевые понятия GitHub Actions сведены в таблицу 1.

Таблица 1 – Ключевые понятия GitHub Actions

Понятие	Описание
Workflow	Автоматизированный процесс, описанный в YAML-файле.
Event	Действие, которое запускает workflow: push, pull_request, workflow_run, schedule, manual dispatch и другие.
Job	Набор шагов, выполняемых на одном runner. Job'ы могут запускаться параллельно или последовательно (через needs).
Step	Отдельный шаг внутри job: либо готовая action (uses), либо команда оболочки (run).
Runner	Виртуальная машина GitHub (ubuntu-latest, windows-latest, macos-latest) или собственный self-hosted runner, на котором выполняется job.
Action	Готовый компонент от сообщества или GitHub, например actions/checkout, astral-sh/setup-uv, codecov/codecov-action.
CI	Непрерывная интеграция: линтинг, форматирование, проверка типов, unit-, integration- и e2e-тесты.
CD	Непрерывная доставка/развёртывание: сборка артефакта, публикация и деплой на сервер.
Pipeline	Последовательность действий, выполняемых при срабатывании триггера, например при push в ветку main.

1.3 Структура workflow-файла

Workflow размещаются в каталоге .github/workflows/ и могут иметь расширение .yml или .yaml. Файл состоит из нескольких верхних уровней:

имени, триггеров, прав, политики параллельного запуска и набора job'ов. Обобщённая структура, использованная в проекте, показана ниже.

```
name:

on:

# права доступа к содержимому репозитория (read, write, none)
permissions:
  contents: read

# параллельный запуск: отменить предыдущий, если пришёл новый
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  job_name:
    needs:
    runs-on:
    timeout-minutes:

    strategy:
      fail-fast:
      matrix:

    services:
      service_name:
        image:
        env:
        ports:
        options:

    steps:
      - name:
        uses:
        with:

      - name:
        run:
```

В исходных заметках верхнеуровневый ключ прав был записан как permission; в GitHub Actions он называется permissions (во множественном числе). Ключ fail-fast также пишется через дефис. Эти детали важны: опечатка

в имени ключа приводит к ошибке валидации workflow или к тому, что настройка молча игнорируется.

Каждый верхний уровень отвечает за свою часть пайплайна:

- name — человекочитаемое имя workflow, отображаемое во вкладке Actions;
- on — события, при которых workflow запускается;
- permissions — минимально необходимые права токена GITHUB_TOKEN;
- concurrency — группа одновременных запусков и политика отмены;
- jobs — словарь job'ов, каждый со своими шагами, стратегией и сервисами.

1.4 Триггеры, права и параллельность

Триггеры задаются в блоке on. В проекте используются два основных события: push и pull_request с фильтром по ветке main. Это означает, что пайплайн запускается при каждом коммите в основную ветку и при каждом обновлении pull request, направленного в неё. Дополнительно можно ограничивать запуск по путям (paths, paths-ignore) или по расписанию (schedule).

Права определяют, что разрешено делать workflow. Принцип наименьших привилегий требует выдавать только необходимый минимум. Для CI, который только читает код и отправляет отчёт о покрытии, достаточно permissions: contents: read. Отдельный токен Codecov передаётся через секрет, а не через права GITHUB_TOKEN.

Параллельность управляется блоком concurrency. Группа ${{ github.workflow }}-${{ github.ref }} объединяет запуски одного workflow в одной ветке, а cancel-in-progress: true отменяет предыдущий запуск при поступлении нового коммита. Это экономит ресурсы и не создаёт очередь устаревших проверок: в истории проекта запуск #2 был отменён именно после нового push.

Для CD дополнительно используется событие workflow_run: оно позволяет запустить развёртывание только после успешного завершения CI. Этот механизм рассматривается в разделе 6.

1.5 Job'ы, шаги, матрица и сервисы

Job — независимая единица работы. По умолчанию job'ы выполняются параллельно, а ключ needs задаёт зависимости и превращает набор job'ов в ориентированный граф. В проекте job tests зависит от check, поэтому дорогие тесты не запускаются, пока не пройдены быстрые проверки качества.

Стратегия матрицы (strategy.matrix) позволяет запустить один и тот же job для нескольких версий Python или операционных систем. Параметр fail-fast: false означает, что падение одной комбинации не отменяет остальные: это важно, когда нужно увидеть полную картину совместимости. В проекте матрица включает Python 3.12, 3.13 и 3.14.

Сервисы (services) — контейнеры, которые запускаются рядом с job и доступны по сети. Они используются для PostgreSQL и Redis. Параметр options позволяет задать healthcheck, чтобы шаги не начали работу, пока база данных не готова принимать подключения.

Шаги бывают двух видов: uses подключает готовую action (например, actions/checkout), а run выполняет команду оболочки. Для передачи параметров action используется блок with.

```
steps:
  - name: Clone Repository
    uses: actions/checkout@v4

  - name: Setup uv
    uses: astral-sh/setup-uv@v7
    with:
      version: '0.12.21'
      enable-cache: true

  - name: Linting
    run: uv run ruff check .
```

1.6 Воспроизводимость и кэширование с uv

Проект использует современный менеджер пакетов uv и файл блокировки uv.lock. Это даёт два важных свойства. Во-первых, зависимости устанавливаются точно в тех версиях, которые зафиксированы в lock-файле. Во-вторых, установка выполняется значительно быстрее, чем через классический pip.

В CI применяется команда uv sync --frozen: она запрещает изменять lock-файл и завершается ошибкой, если pyproject.toml и uv.lock рассогласованы. Это защищает от ситуации «локально работает, а в CI — нет». Action astral-sh/setup-uv с параметром enable-cache: true кэширует загруженные пакеты между запусками, что сокращает время установки.

Версия Python задаётся отдельно командой uv python install. В matrix-job используется переменная ${{ matrix.python-version }}, поэтому каждая комбинация получает свою версию интерпретатора.

2 АНАЛИЗ ПРОЕКТА TASKSERVICE

2.1 Назначение и функциональность

TaskService — production-ready REST API для управления задачами. Проект написан на Python с использованием FastAPI и следует принципам чистой архитектуры. Основные возможности:

- аутентификация по JWT: access- и refresh-токены, подписанные RSA (RS256);
- ролевая модель: администратор, VIP-пользователь и обычный пользователь;
- CRUD-операции над задачами, статусы выполнения и закрытия, лимиты на количество задач;
- аудит действий пользователей и задач;
- управление сессиями и refresh-токенами;
- ограничение частоты запросов (rate limiting) через SlowAPI;
- фоновые задачи Celery: отправка писем и ежедневная очистка;
- наблюдаемость MELT: метрики Prometheus, события Sentry, логи Loguru/Loki, трассировка.

Публичная страница репозитория проекта показана на рисунке 2. В README приведены бейджи CI и Codecov, а также описание технологического стека.

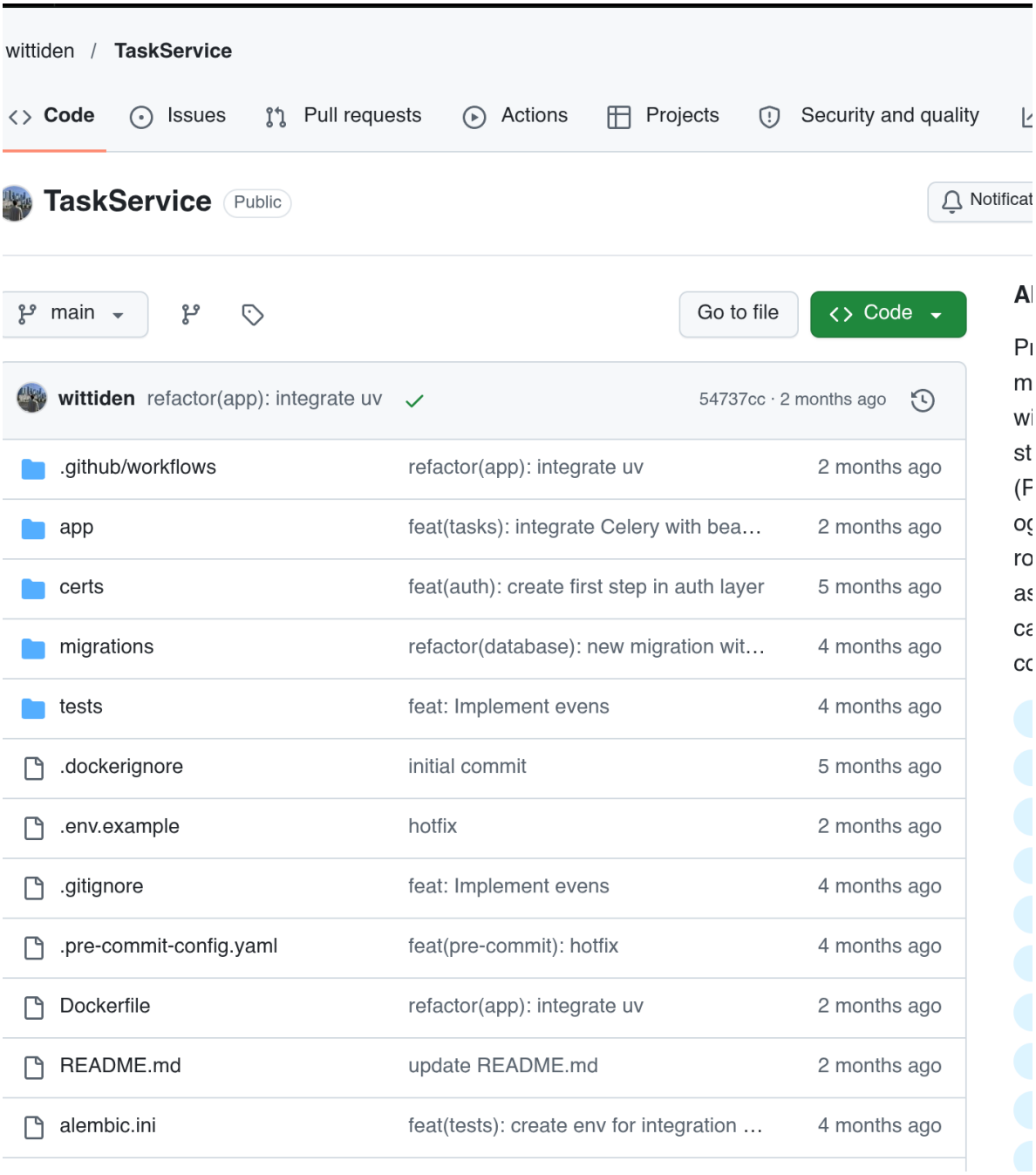


Рисунок 2 – Страница проекта TaskService на GitHub

2.2 Архитектура приложения

Проект разделён на четыре слоя. Presentation Layer содержит FastAPI роутеры, Pydantic-схемы и middleware. Application Layer — use cases и сервисы, в которых сосредоточена бизнес-логика. Domain Layer описывает сущности, value objects и правила. Infrastructure Layer реализует репозитории, Unit of Work, работу с SQLAlchemy, Redis и внешними сервисами. Зависимости направлены внутрь: внешние слои знают о внутренних, но не наоборот.

Связывание компонентов выполняет контейнер внедрения зависимостей Dishka. Репозитории разделены на команды и запросы (CQS), транзакции управляются через Unit of Work, а доступ к PostgreSQL организован асинхронным SQLAlchemy 2.0. Рядом с HTTP-приложением работает слой фоновой обработки: FastAPI публикует задачи в RabbitMQ, worker и beat выполняют их, а результат сохраняется в Redis. Схема архитектуры приведена на рисунке 3.

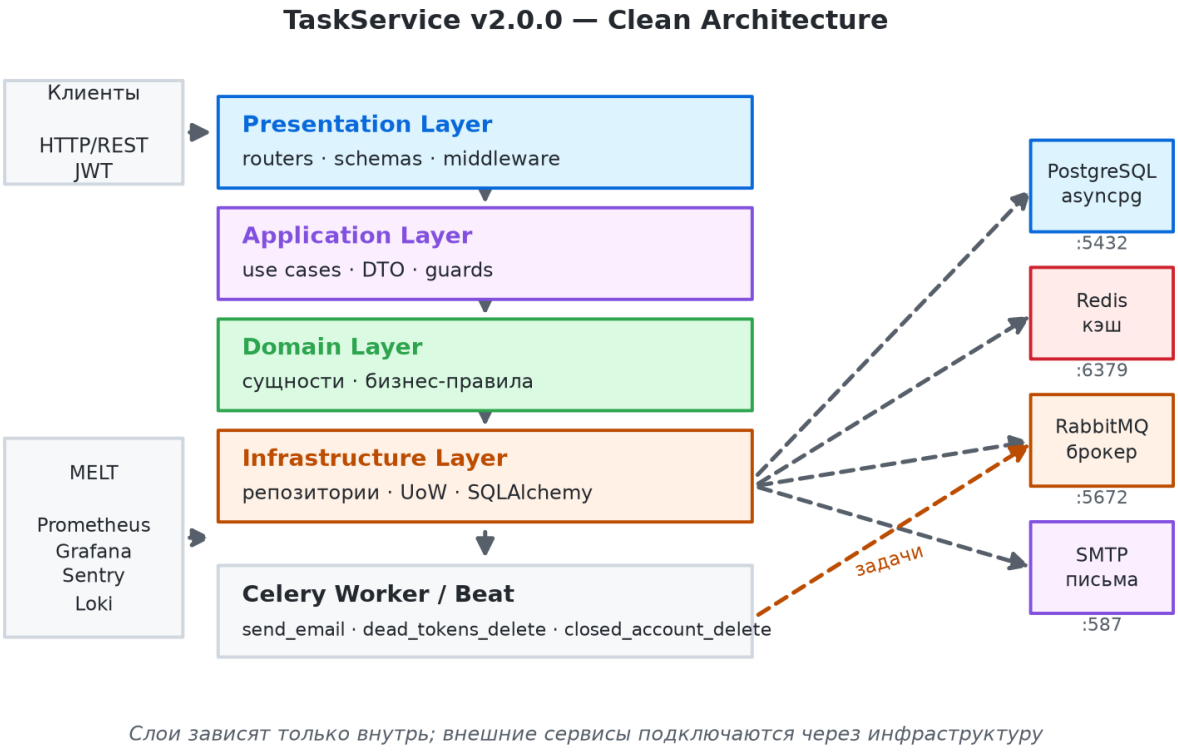


Рисунок 3 – Архитектура TaskService и внешние сервисы

2.3 Структура репозитория

Структура проекта организована по слоям и бизнес-модулям. Ниже приведён сокращённый вид каталогов, существенных для CI:

```
TaskService/
├── .github/
│   └── workflows/
│       └── ci.yaml            # CI-пайплайн
├── app/
│   ├── main.py                # точка входа
│   ├── bootstrap/             # фабрика приложения, middleware, роутеры
│   ├── common/                # конфигурация, безопасность, Celery, MELT
│   ├── container/             # DI-контейнер Dishka
│   ├── infrastructure/        # БД, Redis, HTTP, Unit of Work
│   └── modules/               # auth, users, tasks, sessions, audits
├── tests/
│   ├── unit/                  # unit-тесты (43)
│   ├── integration/           # интеграционные тесты (32)
│   └── factories/             # фабрики тестовых данных
├── migrations/                # миграции Alembic
├── certs/                     # RSA-ключи (не коммитятся)
├── Dockerfile
├── docker-compose.yaml
├── pyproject.toml             # зависимости и настройки
├── pytest.ini                 # маркеры и окружение тестов
├── codecov.yaml               # порог покрытия
├── .pre-commit-config.yaml    # локальные git-хуки
└── uv.lock                    # зафиксированные зависимости
```

Каталог .github/workflows — единственное место, где GitHub ищет workflow. Каталог tests повторяет структуру app, что упрощает навигацию. Файлы pyproject.toml, pytest.ini, codecov.yaml и .pre-commit-config.yaml задают единые правила качества для локальной разработки и CI. В коммите 54737cc проект перешёл с requirements-*.txt на pyproject.toml + uv.lock, и CI был обновлён соответствующим образом.

2.4 Технологический стек и требования к окружению

Для воспроизведения CI важно понимать, какие версии инструментов используются. Основные компоненты приведены в таблице 2.

Таблица 2 – Основные компоненты проекта

Компонент	Версия	Назначение
Python	3.12–3.14	Язык приложения и матрица CI
uv	0.12.21	Установка зависимостей и запуск команд
FastAPI	0.138.2	Веб-фреймворк

Компонент	Версия	Назначение
SQLAlchemy	2.0.51	Асинхронный ORM
PostgreSQL	16	Основная база данных
Redis	8.0	Кэш, сессии, result backend
RabbitMQ	3.8	Брокер сообщений Celery
Celery	5.6.3	Фоновая обработка задач
pytest	9.1.1	Фреймворк тестирования
pytest-xdist	3.8.0	Параллельный запуск тестов (-n auto)
Ruff	0.15.20	Линтинг и форматирование
Pyright	1.1.411	Статическая проверка типов
Codecov	action v5	Отчёт и история покрытия

2.5 Организация тестирования

Тесты — основа CI. В проекте используется pytest с асинхронным режимом asyncio_mode = auto, что позволяет писать async def test_... без дополнительных декораторов. В pytest.ini задано окружение ENVIRONMENT=test, а в pyproject.toml — маркеры unit, integration, e2e, slow, async и параметры покрытия.

Всего в проекте 75 тестов: 43 unit-теста и 32 интеграционных. Unit-тесты проверяют use cases и сервисы с подменой репозиториев через AsyncMock, а интеграционные — реальные HTTP-эндпоинты через httpx.AsyncClient и ASGITransport, с настоящими PostgreSQL и Redis. Распределение тестов по каталогам показано на рисунке 4.

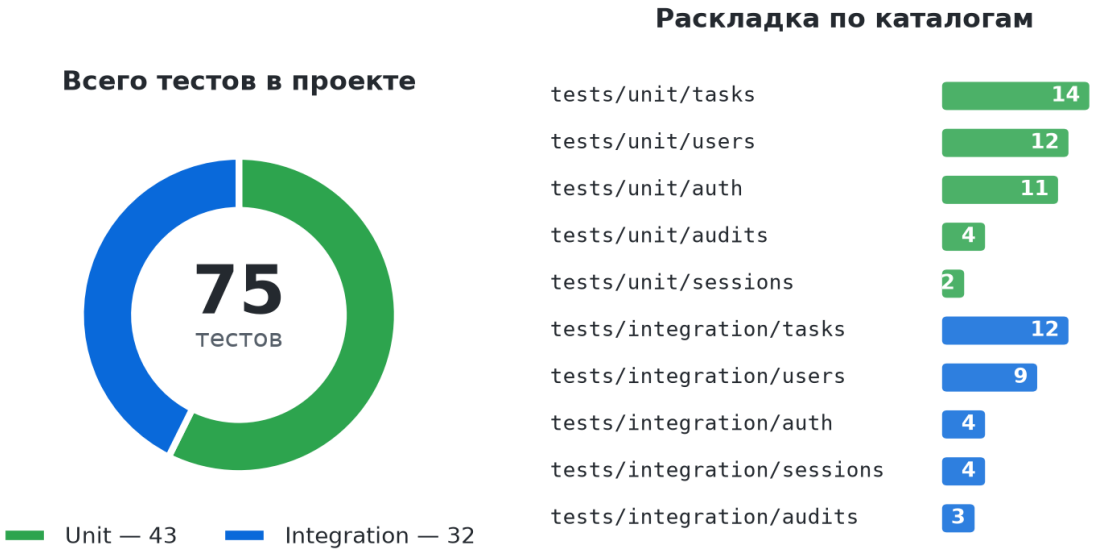


Рисунок 4 – Структура тестов проекта TaskService

Интеграционные тесты используют testcontainers: в conftest.py поднимаются контейнеры postgres:16-alpine и redis:8.0-alpine, после чего к базе применяются миграции Alembic командой upgrade(alembic_config, 'head'). Это гарантирует, что тесты работают с той же схемой БД, что и production. Тестовые данные создаются фабриками factory-boy и Faker.

Покрытие измеряется через pytest-cov. В pyproject.toml задан минимальный порог fail_under = 70, а в CI он дополнительно передаётся флагом --cov-fail-under=70. Если покрытие падает ниже порога, job завершается с ошибкой — это защищает проект от постепенной деградации тестов.

Для запуска тестов в CI используется команда:

```
uv run pytest -m "unit or integration" -n auto
```

Флаг -m выбирает тесты по маркерам, а -n auto (плагин pytest-xdist) распределяет их по всем доступным ядрам runner'a. На этапе с покрытием добавляются отчёты term, html и xml.

2.6 Предпосылки внедрения CI

К моменту внедрения CI проект уже содержал 136 коммитов, несколько бизнес-модулей и 75 тестов. Проверять всё вручную перед каждым коммитом стало дорого: нужно было запустить линтер, форматтер, Pyright, тесты на трёх версиях Python, поднять PostgreSQL и Redis, сгенерировать RSA-ключи. Часть этих действий легко забыть, а часть — выполнить в отличающемся окружении.

Дополнительные предпосылки:

- матрица поддерживаемых версий Python требует повторяемого запуска одних и тех же тестов;
- интеграционные тесты зависят от PostgreSQL и Redis, поэтому окружение должно быть описано декларативно;
- JWT-ключи не хранятся в репозитории и должны генерироваться в CI;
- покрытие нужно отслеживать в динамике, а не только в локальном отчёте;
- статус проверок должен блокировать слияние ошибочного кода в main.

История проекта подтверждает пользу автоматизации: из 32 запусков CI 10 завершились ошибкой на этапе становления пайплайна, и каждая такая ошибка была обнаружена и исправлена следующими коммитами. Без CI эти проблемы обнаружились бы значительно позже.

3 ПРОЕКТИРОВАНИЕ CI-ПАЙПЛАЙНА

3.1 Требования к пайплайну

Перед написанием YAML были сформулированы требования, которым должен удовлетворять пайплайн. Они приведены в таблице 3.

Таблица 3 – Требования к CI-пайплайну

Требование	Реализация в проекте
Быстрая обратная связь	Сначала job check (линтинг, формат, типы), затем тесты
Полнота проверки	Ruff, Ruff format, Pyright, unit- и integration-тесты
Совместимость версий	Матрица Python 3.12, 3.13, 3.14
Реальная инфраструктура	Сервисы PostgreSQL 16 и Redis 8 с healthcheck
Воспроизводимость	uv.lock, uv sync --frozen, фиксированная версия uv
Контроль покрытия	Порог 70 %, отчёты term/html/xml, Codecov
Безопасность	permissions: contents: read, секреты GitHub, генерация ключей
Экономия ресурсов	Кэш uv, concurrency с cancel-in-progress, timeout 10 минут
Наглядность	Имена шагов, бейджи CI и Codecov в README

3.2 Архитектура пайплайна

Пайплайн состоит из двух job'ов. Job check выполняет быстрые проверки: клонирование репозитория, установку uv и Python, установку зависимостей, линтинг, проверку форматирования и статическую проверку типов. Job tests зависит от check и запускается в трёх экземплярах через матрицу. В каждом экземпляре поднимаются PostgreSQL и Redis, генерируются JWT-ключи и выполняются тесты. На версии Python 3.14 дополнительно собирается покрытие и отправляется в Codecov.

Такое разделение даёт быстрый первый сигнал: если в коде синтаксическая ошибка или нарушен стиль, разработчик узнаёт об этом через несколько секунд, не дожидаясь полного прогона тестов. Граф workflow показан на рисунке 5.

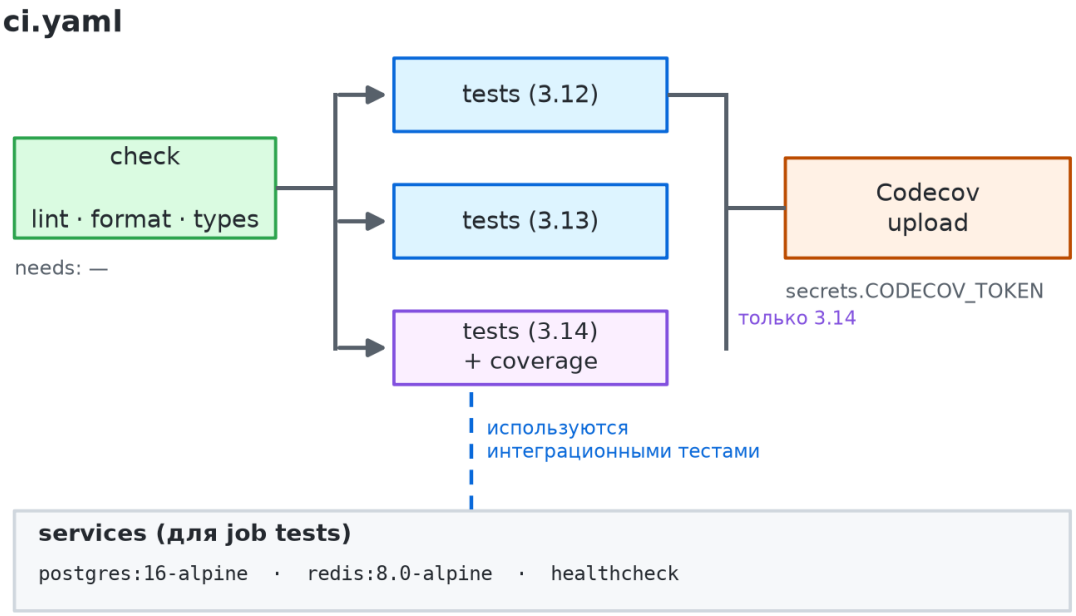


Рисунок 5 – Граф workflow ci.yaml

3.3 Выбор событий и политики запуска

Выбраны события push и pull_request с фильтром по ветке main. Push в основную ветку запускает пайплайн после слияния, а pull_request — до слияния, что позволяет использовать статус проверок как обязательное условие merge. Политика concurrency с cancel-in-progress: true обеспечивает актуальность результата: если в pull request пришёл новый коммит, старый запуск отменяется.

Фильтр по веткам важен и с точки зрения ресурсов: запуск на каждый push в любую ветку мог бы расходовать минуты GitHub Actions без необходимости. При этом pull request в main всё равно проверяется, даже если работа ведётся в отдельной ветке.

3.4 Разделение на job'ы и кэширование

Job'ы изолированы друг от друга: у каждого свой runner и своя файловая система. Поэтому каждый job заново клонирует репозиторий и устанавливает зависимости. Кэш setup-uv компенсирует часть накладных расходов, а timeout-minutes: 10 защищает от зависших процессов.

Матрица из трёх версий Python увеличивает общее время пайплайна, но запускается параллельно: на практике job'ы tests (3.12), tests (3.13) и tests (3.14)

выполняются одновременно. Именно поэтому общее время последнего успешного запуска составило около двух минут, а не сумму трёх прогонов.

3.5 Работа с секретами

В репозитории не хранятся токен Codecov и приватные ключи JWT. Токен Codecov добавляется в настройках GitHub: Settings → Secrets and variables → Actions → New repository secret под именем CODECOV_TOKEN. В workflow он доступен как ${{ secrets.CODECOV_TOKEN }} и не выводится в логи.

RSA-ключи генерируются заново на каждом запуске в каталоге certs. Это означает, что тесты проверяют реальный механизм подписи и проверки токенов, но приватные ключи не попадают в артефакты и историю Git. Каталог certs исключён из репозитория через .gitignore.

4 РЕАЛИЗАЦИЯ WORKFLOW CI.YAML

4.1 Заголовок workflow и политика запуска

Файл начинается с имени, триггеров, прав и политики параллельности. Именно эти несколько строк определяют, когда и с какими правами будет работать пайплайн:

```
name: CI

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

permissions:
  contents: read

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

name: CI задаёт отображаемое имя. Блок on включает запуск при push и pull request в main. permissions: contents: read оставляет токену только чтение

кода — этого достаточно для checkout и запуска тестов. concurrency группирует запуски по workflow и ref, а cancel-in-progress отменяет устаревшие.

4.2 Job check: линтинг, форматирование и типы

Job check выполняет статические проверки. Он не зависит от других job'ов и запускается первым:

```
check:
  runs-on: ubuntu-latest
  timeout-minutes: 10

  steps:
    - name: Clone Repository
      uses: actions/checkout@v4

    - name: Setup uv
      uses: astral-sh/setup-uv@v7
      with:
        version: '0.12.21'
        enable-cache: true

    - name: Setup Python
      run: uv python install 3.14

    - name: Install Dependencies
      run: uv sync --frozen

    - name: Linting
      run: uv run ruff check .

    - name: Formatting
      run: uv run ruff format . --check

    - name: Type Checking
      run: uv run pyright
```

Разберём шаги подробнее:

- Clone Repository — actions/checkout@v4 клонирует репозиторий в рабочую директорию runner'a;
- Setup uv — устанавливает uv версии 0.12.21 и включает кэш пакетов;

- Setup Python — ставит Python 3.14 через uv (в job check достаточно одной версии);
- Install Dependencies — uv sync --frozen ставит зависимости строго по uv.lock;
- Linting — ruff check . ищет ошибки стиля и потенциальные проблемы;
- Formatting — ruff format . --check проверяет форматирование без изменения файлов;
- Type Checking — pyright выполняет статический анализ типов в строгом режиме.

Параметр timeout-minutes: 10 ограничивает время job'a. Если установка зависимостей или проверка зависли, job будет остановлен, а не будет расходовать минуты до общего лимита.

4.3 Job tests: матрица и сервисы

Job tests начинается с зависимости от check, матрицы версий и описания сервисов:

```
tests:
  needs: check
  runs-on: ubuntu-latest
  timeout-minutes: 10

  strategy:
    fail-fast: false
    matrix:
      python-version: ['3.12', '3.13', '3.14']

  services:
    postgres:
      image: postgres:16-alpine
      env:
        POSTGRES_USER: postgres
        POSTGRES_PASSWORD: root
        POSTGRES_DB: test_postgres
      ports:
        - 5432:5432
      options: >-
        --health-cmd pg_isready
        --health-interval 10s
        --health-timeout 5s
        --health-retries 5

    redis:
      image: redis:8.0-alpine
      ports:
        - 6379:6379
      options: >-
        --health-cmd "redis-cli ping"
        --health-interval 10s
        --health-timeout 5s
        --health-retries 5
```

needs: check гарантирует, что тесты не начнутся, пока не пройдены лингинг и типы. fail-fast: false не отменяет остальные версии Python при падении одной — это позволяет увидеть, например, что код работает на 3.14, но не работает на 3.12.

Сервисы postgres и redis описывают контейнеры, которые GitHub запускает до выполнения шагов. Healthcheck'и pg_isready и redis-cli ping не дают шагам начать работу, пока базы не готовы. Порты проброшены на localhost runner'a, поэтому приложение может подключаться к ним по стандартным адресам.

4.4 Генерация JWT-ключей

Приложение использует RSA-подпись токенов. Приватные ключи не хранятся в репозитории, поэтому в CI они создаются перед тестами:

```
- name: Generate JWT Keys
  run: |
    cd certs
    openssl genrsa -out access-private.pem 2048
    openssl genrsa -out refresh-private.pem 2048
    openssl rsa -in access-private.pem -pubout -out access-public.pem
    openssl rsa -in refresh-private.pem -pubout -out refresh-public.pem
```

Команды создают два приватных ключа и соответствующие им публичные. Каталог certs должен существовать в репозитории (в нём лежит README.md), иначе cd certs завершится ошибкой — именно такой сбой произошёл в запуске #9, после чего шаг был скорректирован.

Генерация ключей на каждом запуске — компромисс между безопасностью и простотой. Для production ключи хранят в секретах или в специализированном хранилище, а в CI генерируют временные, чтобы не передавать приватный ключ на runner.

4.5 Запуск тестов и сбор покрытия

Шаги тестирования различаются для версии 3.14 и остальных версий:

```
- name: Run Unit & Integration Tests
  if: matrix.python-version != '3.14'
  run: uv run pytest -m "unit or integration" -n auto

- name: Run Unit & Integration Tests & Coverage
  if: matrix.python-version == '3.14'
  run: >
    uv run pytest -m "unit or integration" -n auto
    --cov=app --cov-fail-under=70
    --cov-report=term
    --cov-report=html
    --cov-report=xml
```

На версиях 3.12 и 3.13 тесты выполняются без сбора покрытия — это экономит время. На 3.14 добавляются флаги --cov=app, порог --cov-fail-under=70 и три формата отчёта: term для логов, html для просмотра и xml для

Codecov. Условие if использует контекст матрицы, поэтому шаг выполняется только в одном экземпляре job'a.

Флаг -n auto включает pytest-xdist и распределяет тесты по ядрам. Для интеграционных тестов это особенно важно: каждый тест работает с собственными данными, а контейнеры поднимаются один раз на сессию.

В pytest.ini и pyproject.toml уже заданы базовые параметры покрытия, поэтому локальный запуск uv run pytest также проверяет порог 70 %. Дублирование флага в CI делает требование явным и не зависящим от локальных настроек.

4.6 Публикация покрытия в Codecov

Последний шаг job'a отправляет отчёт в Codecov:

```
- name: Upload coverage reports to Codecov
  if: matrix.python-version == '3.14'
  uses: codecov/codecov-action@v5
  with:
    token: ${{ secrets.CODECOV_TOKEN }}
    fail_ci_if_error: true
```

fail_ci_if_error: true делает загрузку отчёта обязательной: если Codecov недоступен или токен неверен, пайплайн завершится ошибкой. Это осознанный выбор: отчёт о покрытии является частью качества, а не необязательным украшением.

Файл codecov.yaml задаёт целевое покрытие проекта 70 % и отключает проверку patch-покрытия:

```
coverage:
  status:
    project:
      default:
        target: 70%
    patch: false
```

История покрытия и бейдж доступны на странице Codecov. Бейдж в README обновляется автоматически и показывает текущий процент.

4.7 Эволюция пайплайна и типичные ошибки

Пайплайн не появился сразу в финальном виде. В истории проекта видно, как он дорабатывался: сначала был только job с линтингом и типами, затем добавились unit- и integration-тесты, потом матрица версий, покрытие и Codecov. В таблице 4 собраны реальные неудачные запуски и шаги, на которых они остановились.

Таблица 4 – Неудачные запуски CI и их причины

Запуск	Коммит	Шаг с ошибкой	Что показал запуск
#1	feat(workflows): add ci pipeline	Run tests	Первый запуск: шаг тестов завершился ошибкой, потребовалась доработка workflow.
#2	refactor(ci): add ci only with lint, format, check	check (cancelled)	Запуск отменён новым коммитом — сработала политика concurrency.
#3	hotfix ubuntu version	Type Checking	Проверка типов выявила несовместимость с выбранной версией окружения.
#5, #6	feat(ci): hotfix check job; add default params	Start Unit Tests	Не хватало зависимостей/настроек, необходимых для запуска unit-тестов.
#8, #10	feat(ci): add integration job; hotfix mrdir in ci	Integration Tests	Интеграционные тесты не могли поднять окружение; исправлен каталог и подготовка сервисов.
#9	hotfix	Generate JWT Keys	Шаг генерации ключей не находил каталог certs.
#14	refactor(ci): global optimization	Unit & Integration Tests (3.12, 3.13)	Код использовал возможности, доступные только в Python 3.14; добавлены совместимые аннотации.
#21	feat: Implement evens	Run Unit & Integration Tests	Регрессия после изменения бизнес-логики; исправлена следующим коммитом.
#25	hotfix toml	Linting	Ошибка конфигурации/стиля, остановившая job check за 35 секунд.

Анализ таблицы позволяет сформулировать практические выводы. Во-первых, быстрые проверки должны идти первыми: запуск #25 остановился на линтинге через 35 секунд, не тратя время на тесты. Во-вторых, ошибки окружения (каталоги, ключи, сервисы) — самая частая причина сбоев на старте; их лучше проверять локально теми же командами. В-третьих, матрица версий выявляет несовместимости, которые незаметны на одной версии Python.

5 АНАЛИЗ РЕЗУЛЬТАТОВ РАБОТЫ ПАЙПЛАЙНА

5.1 Статистика запусков

За период с 29 июля по 1 октября 2026 года workflow CI запускался 32 раза. Сводная статистика приведена в таблице 5. Данные получены из GitHub API и интерфейса Actions.

Таблица 5 – Статистика запусков workflow CI

Показатель	Значение
Всего запусков CI	32
Успешных	21 (65,6 %)
С ошибкой	10 (31,3 %)
Отменённых	1 (3,1 %)
Средняя длительность успешного запуска	127 с
Медианная длительность	123 с
Минимальная / максимальная длительность	35 с / 213 с
Период наблюдения	29.07.2026 – 01.10.2026

История запусков в интерфейсе GitHub Actions показана на рисунке 6. Зелёные отметки соответствуют успешным запускам, красные — ошибкам. Видно, что после стабилизации пайплайна (сентябрь–октябрь) запуски завершаются успешно и укладываются примерно в две минуты.

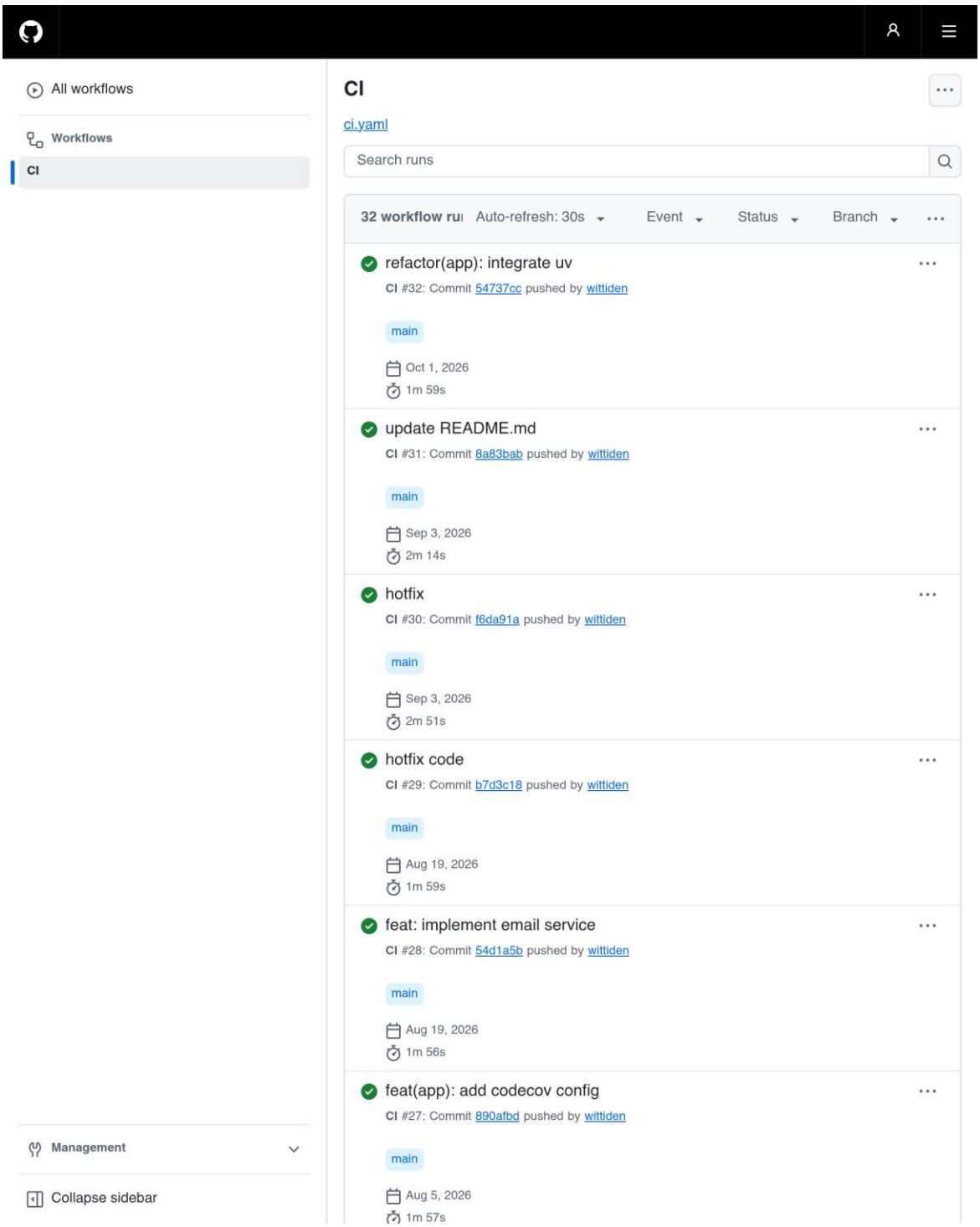


Рисунок 6 – История запусков workflow CI в GitHub Actions

Распределение длительности последних 18 запусков приведено на рисунке 7. Большинство успешных запусков занимает 110–145 секунд. Два всплеска объясняются особенностями этапа: запуск #30 длился 171 с, а запуск #24 — 143 с. Неудачные запуски часто короче, потому что останавливаются на первом же проблемном шаге.



Рисунок 7 – Длительность последних запусков CI

5.2 Успешный запуск #32

Последний успешный запуск #32 (коммит 54737cc, «refactor(app): integrate uv») завершился за 1 минуту 59 секунд. На странице запуска видны четыре job'a: check и три экземпляра tests для версий 3.12, 3.13 и 3.14 (рисунок 8).

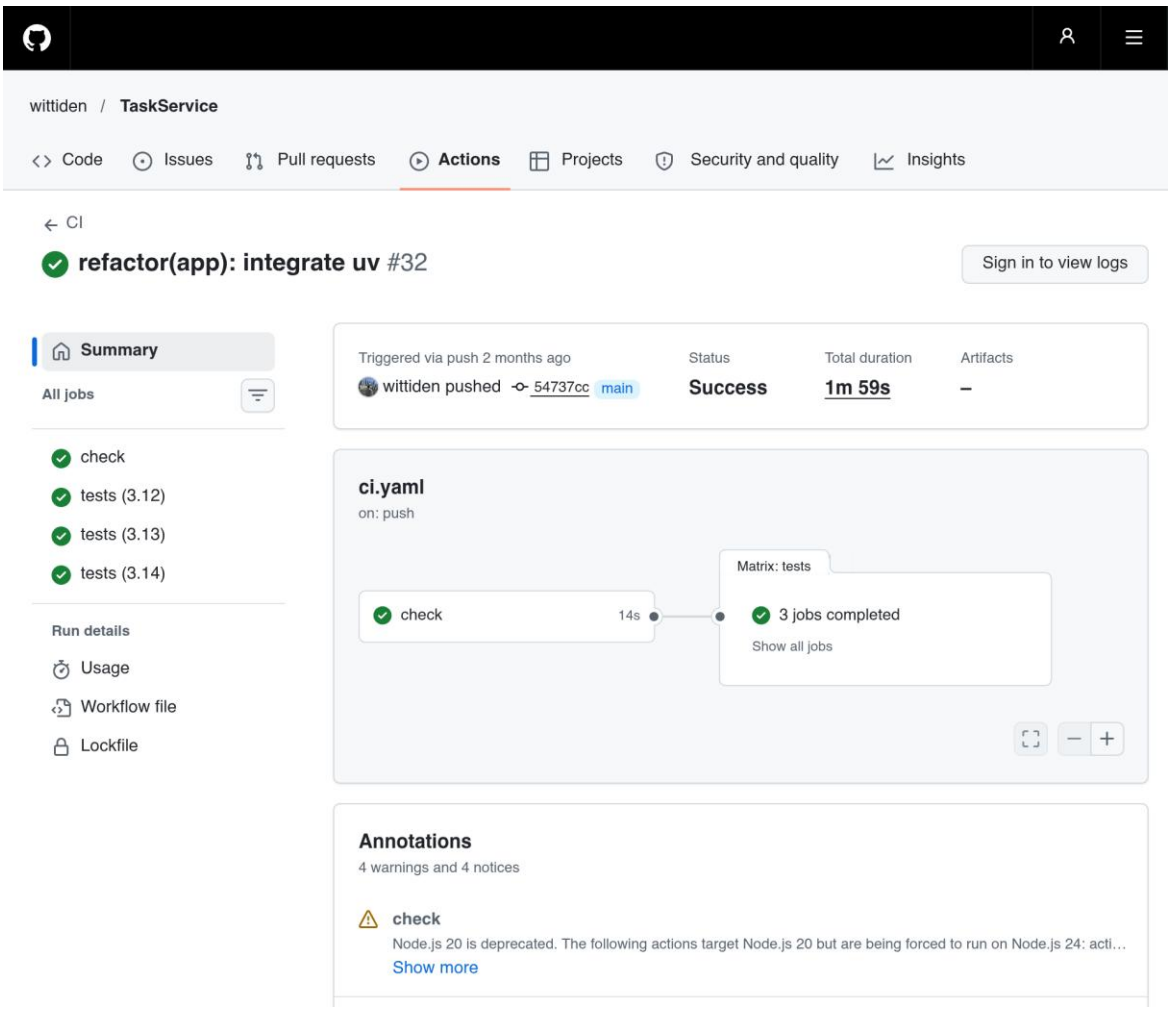


Рисунок 8 – Успешный запуск #32 workflow CI

Длительности job'ов приведены на рисунке 9: check — 14 с, tests (3.12) — 61 с, tests (3.13) — 52 с, tests (3.14) — 64 с. Job'ы тестов выполнялись параллельно, поэтому общее время определяется самым медленным из них, а не суммой.

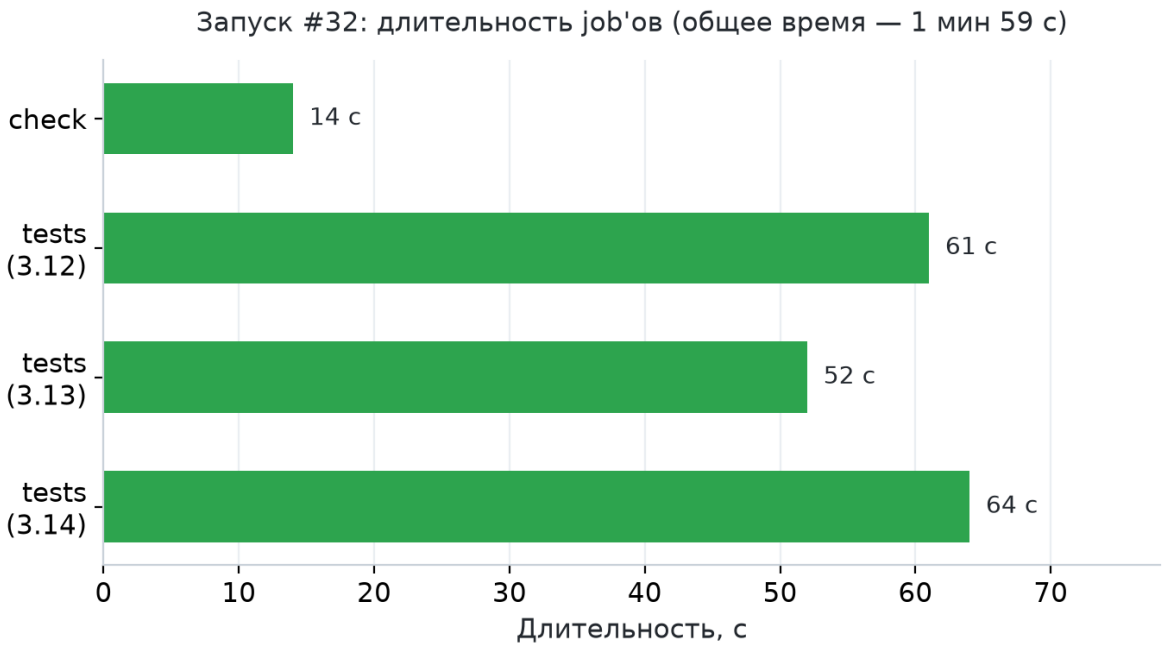


Рисунок 9 – Длительность job'ов успешного запуска

На странице запуска также отображаются предупреждения: actions/checkout@v4 и другие action'ы используют Node.js 20, который постепенно вытесняется Node.js 24, а метка ubuntu-latest с октября 2026 года мигрирует на Ubuntu 26. Эти предупреждения не влияют на результат, но показывают, что workflow нужно регулярно обновлять. Практическая рекомендация — перейти на actions/checkout@v5 и следить за заметками о релизах используемых action'ов.

5.3 Анализ сбоев

Неудачные запуски — не недостаток, а источник информации. На рисунке 10 показан запуск #25 («hotfix toml»), который завершился ошибкой за 35 секунд: job check упал на шаге линтинга, а job tests был пропущен из-за зависимости needs.

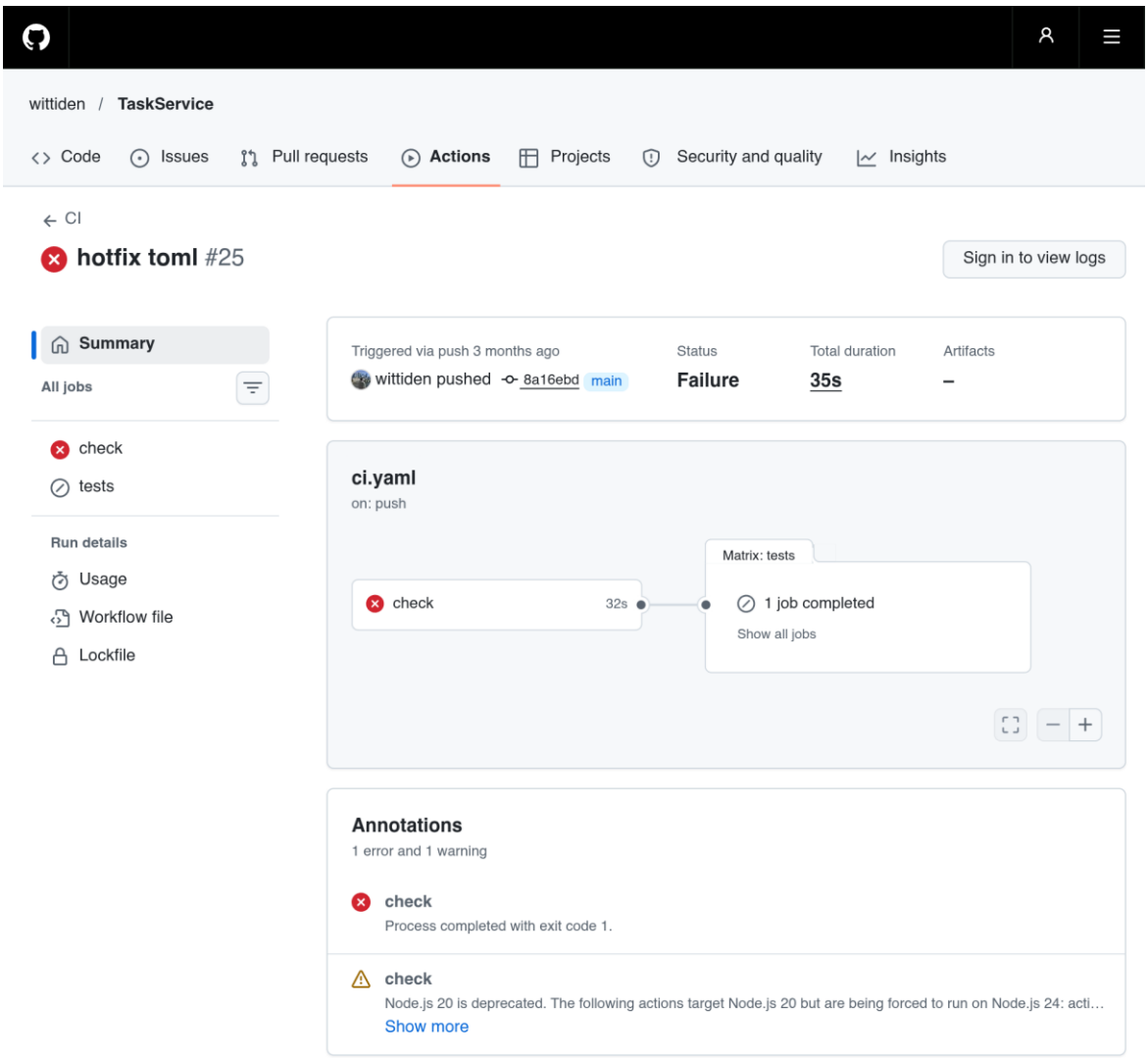


Рисунок 10 – Неудачный запуск #25: ошибка на шаге линтинга

Такое поведение пайплайна правильно: быстрая проверка остановила дорогостоящие тесты, а разработчик получил точный сигнал о проблеме. Основные категории сбоев, выявленные по истории проекта:

- окружение и подготовка — отсутствующие каталоги, ключи, сервисы (запуски #5, #6, #8, #9, #10);
- несовместимость версий — код, работающий только на Python 3.14 (запуск #14);
- качество кода — ошибки линтинга и типов (запуски #3, #25);
- регрессии — падение тестов после изменения бизнес-логики (запуск #21);
- отмена — ожидаемое поведение concurrency при новом коммите (запуск #2).

Для каждой категории можно предложить профилактическую меру: локальный прогон тех же команд, фиксация версий в matrix, pre-commit, а также обязательный статус проверок в настройках ветки. Важно, чтобы

исправление ошибки сопровождалось повторным запуском и зелёным статусом.

5.4 Покрытие и Codecov

По данным Codecov на момент последнего запуска проект имеет 90,65 % покрытия: проанализирован 81 файл, 1947 строк кода, из которых 1765 покрыты тестами, а 182 не покрыты. Это существенно выше минимального порога 70 %, заданного в CI (рисунок 11).



Рисунок 11 – Покрытие проекта по данным Codecov

Высокое покрытие не гарантирует отсутствие ошибок, но вместе с интеграционными тестами и статическим анализом оно даёт хороший уровень уверенности. Порог 70 % оставляет пространство для манёвра, не заставляя команду писать тесты на каждую служебную строку. При этом patch: false означает, что Codecov не блокирует PR из-за недостаточного покрытия новых строк — это возможное направление ужесточения политики.

5.5 Оценка эффективности и рекомендации

Пайплайн обеспечивает полный цикл проверки примерно за две минуты. Основные факторы эффективности:

- разделение на быстрый check и параллельные tests;
- кэширование uv и фиксированные версии зависимостей;
- отмена устаревших запусков через concurrency;

- единые команды локально и в CI, что упрощает воспроизведение ошибок.

Направления развития:

1. Закрепить сторонние action'ы по SHA, а не по тегу, чтобы исключить подмену кода action'a.
2. Обновить actions/checkout до v5 и убрать предупреждения о Node.js 20.
3. Добавить Dependabot для github-actions и uv, чтобы обновления приходили автоматически.
4. Публиковать HTML-отчёт покрытия как артефакт job'a для просмотра без Codecov.
5. Включить обязательные проверки (check, tests (3.12–3.14)) в защите ветки main.
6. Рассмотреть paths-ignore для документации, чтобы не запускать полный пайплайн на изменения README.
7. Включить patch-проверку Codecov, когда команда будет готова к более строгой политике.

6 CD: НЕПРЕРЫВНАЯ ДОСТАВКА И РАЗВЁРТЫВАНИЕ

6.1 От CI к CD

CI отвечает на вопрос «можно ли доверять коду». CD отвечает на вопрос «как доставить проверенный код в среду выполнения». Между ними находится сборка артефакта: для Python-приложения это обычно Docker-образ, который публикуется в реестре (GitHub Container Registry, Docker Hub или приватный registry).

Continuous Delivery предполагает, что артефакт всегда готов к развёртыванию, но решение о выпуске принимает человек. Continuous Deployment автоматически разворачивает каждое успешное изменение. Для учебного проекта достаточно первого варианта: автоматическая сборка и публикация образа, а деплой — по подтверждению или на тестовый стенд.

6.2 CD-workflow проекта

Файл cd.yaml запускался после успешного CI через событие workflow_run и выполнял развёртывание на self-hosted runner:

```yaml
name: CD

on:
  workflow_run:
    workflows:
      - CI
    types:
      - completed
    conclusions:
      - success

permissions:
  contents: read

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  deploy:
    runs-on: self-hosted
    timeout-minutes: 10

    steps:
      - name: Clone Repository
        uses: actions/checkout@v4

      - name: Run Docker Compose
        run: docker compose up -d --build --remove-orphans

      - name: Application Healthcheck
        run: curl -f http://localhost:8000/api/v1/health/app

      - name: Database Healthcheck
        run: curl -f http://localhost:8000/api/v1/health/db

      - name: Redis Healthcheck
        run: curl -f http://localhost:8000/api/v1/health/redis

      - name: Docker logs
        if: failure()
        run: docker compose logs --tail=200
```

Схема workflow показана на рисунке 12. Она содержит правильные элементы: запуск только после успешного CI, сборку через Docker Compose, проверку healthcheck-эндпоинтов и вывод логов при ошибке.

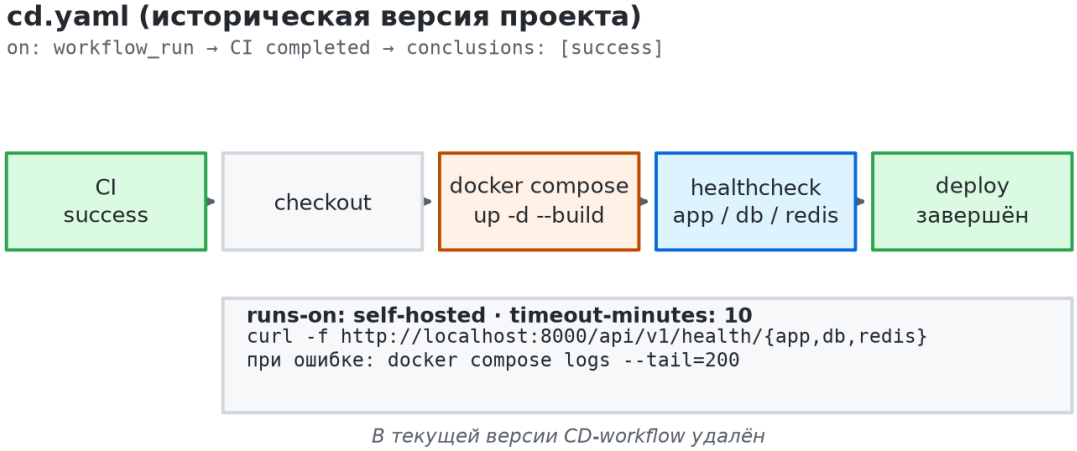


Рисунок 12 – CD-workflow проекта

6.3 Почему CD требует больше, чем CI

CI работает на эфемерной виртуальной машине GitHub, которую можно испортить без последствий. CD воздействует на реальную среду, поэтому требует дополнительных гарантий:

- стабильный runner или сервер с Docker, доступом к registry и сети;
- секреты для registry, SSH, базы данных и внешних сервисов;
- миграции БД (Alembic) с возможностью отката;
- healthcheck и smoke-тесты после развёртывания;
- стратегия отката на предыдущий образ;
- среды (staging, production) и ручное подтверждение для production;
- аудит и уведомления о результате деплоя.

Кроме того, self-hosted runner — это доверенная машина, которая получает код и секреты. Если runner доступен из интернета, необходимо ограничить круг лиц, имеющих право запускать на нём workflow, и не выполнять непроверенный код из pull request'ов.

6.4 Рекомендуемая схема CD

Для TaskService можно рекомендовать схему, приведённую на рисунке 13: сборка и публикация образа в GHCR, затем развёртывание на целевом сервере с проверкой здоровья и возможностью отката.

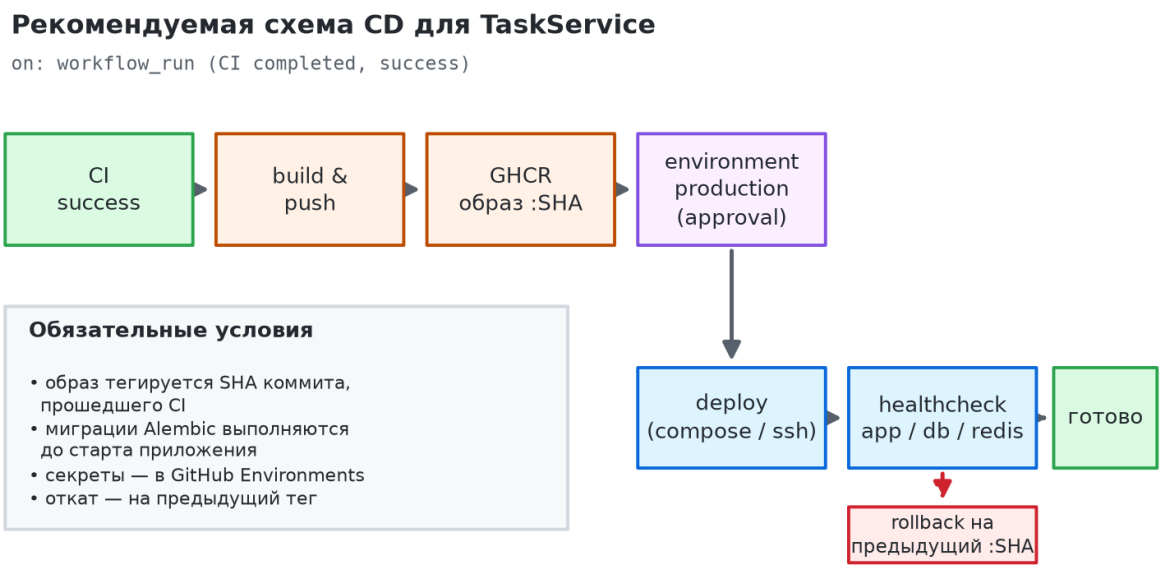


Рисунок 13 – Рекомендуемая схема CD для проекта

Фрагмент workflow для сборки и публикации образа:

```
name: CD

on:
  workflow_run:
    workflows: [CI]
    types: [completed]
    branches: [main]

permissions:
  contents: read
  packages: write

jobs:
  build:
    if: github.event.workflow_run.conclusion == 'success'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
        with:
          ref: ${{ github.event.workflow_run.head_sha }}

      - uses: docker/setup-buildx-action@v3

      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ghcr.io/${{ github.repository }}:${{
github.event.workflow_run.head_sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

Для деплоя рекомендуется отдельный job с environment: production, защищённым обязательным ревью, и шагами:

1. получить образ с тегом SHA из GHCR;
2. выполнить alembic upgrade head в контролируемом окружении;
3. обновить контейнеры (docker compose pull и docker compose up -d);
4. проверить /api/v1/health/app, /db и /redis;

5. при ошибке вернуть предыдущий тег и уведомить команду.

Важно, чтобы деплой использовал тот же коммит, который прошёл CI. Поэтому в checkout передаётся github.event.workflow_run.head_sha, а образ тегируется этим SHA, а не плавающим latest.

6.5 Секреты и безопасность CD

Для CD минимально необходимы следующие секреты и настройки:

Таблица 6 – Секреты и настройки для CD

Секрет/настройка	Назначение
CODECOV_TOKEN	Загрузка отчёта о покрытии (CI)
GITHUB_TOKEN	Публикация образа в GHCR (права packages: write)
SSH_KEY / DEPLOY_KEY	Доступ к серверу развёртывания
ENV_FILE	Переменные окружения production
Environment: production	Ручное подтверждение и защита деплоя

Общие правила: не выводить секреты в логи, использовать ${{ secrets.* }}, ограничивать permissions, закреплять сторонние action'ы по SHA, использовать GitHub Environments с обязательным ревью и не запускать деплой из pull request'ов без доверия.

7 ЛОКАЛЬНОЕ ВОСПРОИЗВЕДЕНИЕ И СОПРОВОЖДЕНИЕ ПАЙПЛАЙНА

7.1 Команды, совпадающие с CI

Главное правило: локальные команды должны совпадать с командами в workflow. Тогда ошибку можно воспроизвести до push. Базовый набор:

```
# установить зависимости строго по lock-файлу
uv sync --frozen

# линтинг и форматирование
uv run ruff check .
uv run ruff format . --check

# автоисправление
uv run ruff check . --fix
uv run ruff format .

# проверка типов
uv run pyright

# тесты и покрытие
uv run pytest -m "unit or integration" -n auto
uv run pytest --cov=app --cov-fail-under=70 --cov-report=term-missing

# установка и запуск pre-commit
uv run pre-commit install
uv run pre-commit run --all-files
```

Для интеграционных тестов требуются Docker и доступные порты. Проект поднимает контейнеры через testcontainers, поэтому отдельный docker compose up для тестов не обязателен, но Docker должен быть запущен. Для ручной проверки приложения используется docker compose up -d.

JWT-ключи для локального запуска генерируются теми же командами OpenSSL, что и в CI (см. раздел 4.4). Если ключей нет, приложение не сможет подписывать и проверять токены.

7.2 Pre-commit как первый барьер

Файл .pre-commit-config.yaml подключает проверки, которые выполняются перед каждым коммитом:

Таблица 7 – Хуки pre-commit в проекте

Hook	Что проверяет
check-yaml	Синтаксис YAML-файлов
check-toml	Синтаксис TOML-файлов
check-json	Синтаксис JSON-файлов
debug-statements	Забытые breakpoint() и pdb
ruff-check	Линтинг с автоисправлением

Hook	Что проверяет
ruff-format	Форматирование кода
pyright	Статическая проверка типов

Pre-commit не заменяет CI, но сокращает количество «красных» запусков: большинство ошибок стиля и типов отлавливается ещё до отправки коммита. В CI те же проверки выполняются обязательно, поэтому обойти локальные хуки не получится.

7.3 Обновление зависимостей и action'ов

Версии action'ов и инструментов нужно обновлять осознанно. Для зависимостей Python используется uv lock --upgrade, после чего обновлённый uv.lock коммитится и проверяется в CI. Для action'ов рекомендуется Dependabot с экосистемой github-actions: он создаёт pull request с новой версией, а пайплайн проверяет совместимость.

Не стоит использовать плавающие теги вроде @main или @latest: они делают сборку невоспроизводимой и создают риск компрометации. Предпочтительны мажорные теги (@v4, @v5) или, для максимальной безопасности, полный SHA коммита.

7.4 Чек-лист при изменении workflow

Перед коммитом изменений в .github/workflows полезно пройти по чек-листу:

1. YAML синтаксически корректен, отступы и двоеточия соблюдены.
2. Триггеры соответствуют задаче: не запускается лишний пайплайн.
3. permissions минимальны, секреты не попали в файл.
4. Версии action'ов зафиксированы, нет @main и @latest.
5. Указаны timeout-minutes, кэш и разумная матрица.
6. Сервисы имеют healthcheck, порты не конфликтуют.
7. Команды локально воспроизводятся без изменений.
8. Проверки добавлены в защиту ветки как обязательные.
9. README и бейджи отражают актуальное состояние.
10. Для CD предусмотрены откат, healthcheck и защита environment.

ЗАКЛЮЧЕНИЕ

В ходе работы был проанализирован реальный проект TaskService, его архитектура, структура, тесты и workflow GitHub Actions. На основе этого анализа построен и подробно разобран CI/CD-пайплайн, который автоматически проверяет стиль, форматирование, типы, unit- и integration-тесты на трёх версиях Python, измеряет покрытие и публикует отчёт в Codecov.

Практическая часть подтвердила основные принципы CI/CD: быстрые проверки должны выполняться раньше дорогих, окружение должно быть воспроизводимым, а результат — наглядным. Анализ 32 реальных запусков показал, что пайплайн прошёл путь от первых ошибок конфигурации до стабильной работы примерно за две минуты. Покрытие проекта составляет 90,65 %, что превышает заданный порог 70 %.

Отдельно рассмотрен переход к CD. Исторический cd.yaml проекта показал принципиальную схему (workflow_run → Docker Compose → healthcheck), но также и ограничения: без подготовленного runner'a и инфраструктуры деплой невозможен. Предложенная схема с публикацией образа в GHCR, защищённым environment и откатом может стать следующим шагом развития проекта.

Полученные навыки универсальны: структура workflow, работа с матрицей, сервисами, кэшем, секретами и отчётами применима к любому серверному проекту на GitHub, а не только к TaskService.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. GitHub Actions: документация. — URL: https://docs.github.com/actions (дата обращения: 08.11.2026).

2. Workflow syntax for GitHub Actions. — URL: https://docs.github.com/actions/reference/workflow-syntax-for-github-actions (дата обращения: 08.11.2026).

3. GitHub Actions: events that trigger workflows. — URL: https://docs.github.com/actions/using-workflows/events-that-trigger-workflows (дата обращения: 08.11.2026).

4. uv: an extremely fast Python package manager. — URL: https://docs.astral.sh/uv/ (дата обращения: 08.11.2026).

5. astral-sh/setup-uv: GitHub Action. — URL: https://github.com/astral-sh/setup-uv (дата обращения: 08.11.2026).

6. Codecov: documentation. — URL: https://docs.codecov.com/ (дата обращения: 08.11.2026).

7. pytest documentation. — URL: https://docs.pytest.org/ (дата обращения: 08.11.2026).

8. Testcontainers for Python. — URL: https://testcontainers-python.readthedocs.io/ (дата обращения: 08.11.2026).

9. FastAPI documentation. — URL: https://fastapi.tiangolo.com/ (дата обращения: 08.11.2026).

10. Репозиторий TaskService. — URL: https://github.com/wittiden/TaskService (дата обращения: 08.11.2026).

11. Внутренние заметки: «Основы GitHub Actions», «Команды GitHub Actions», «GitHub Actions» (личное хранилище Obsidian).

ПРИЛОЖЕНИЕ А. ПОЛНЫЙ ФАЙЛ CI.YAML

Ниже приведён полный текст файла .github/workflows/ci.yaml, используемого в проекте.

```
name: CI

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

permissions:
  contents: read

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  check:
    runs-on: ubuntu-latest
    timeout-minutes: 10

    steps:
      - name: Clone Repository
        uses: actions/checkout@v4

      - name: Setup uv
        uses: astral-sh/setup-uv@v7
        with:
          version: '0.12.21'
          enable-cache: true

      - name: Setup Python
        run: uv python install 3.14

      - name: Install Dependencies
        run: uv sync --frozen

      - name: Linting
        run: uv run ruff check .
```

```
    - name: Formatting
      run: uv run ruff format . --check

    - name: Type Checking
      run: uv run pyright

  tests:
    needs: check
    runs-on: ubuntu-latest
    timeout-minutes: 10

    strategy:
      fail-fast: false
      matrix:
        python-version: ['3.12', '3.13', '3.14']

    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_USER: postgres
          POSTGRES_PASSWORD: root
          POSTGRES_DB: test_postgres
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

      redis:
        image: redis:8.0-alpine
        ports:
          - 6379:6379
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - name: Clone Repository
        uses: actions/checkout@v4
```

```
      - name: Setup uv
        uses: astral-sh/setup-uv@v7
        with:
          version: '0.12.21'
          enable-cache: true

      - name: Setup Python
        run: uv python install ${{ matrix.python-version }}

      - name: Install Dependencies
        run: uv sync --frozen

      - name: Generate JWT Keys
        run: |
          cd certs
          openssl genrsa -out access-private.pem 2048
          openssl genrsa -out refresh-private.pem 2048
          openssl rsa -in access-private.pem -pubout -out access-public.pem
          openssl rsa -in refresh-private.pem -pubout -out refresh-public.pem

      - name: Run Unit & Integration Tests
        if: matrix.python-version != '3.14'
        run: uv run pytest -m "unit or integration" -n auto

      - name: Run Unit & Integration Tests & Coverage
        if: matrix.python-version == '3.14'
        run: uv run pytest -m "unit or integration" -n auto --cov=app --cov-fail-
under=70 --cov-report=term --cov-report=html --cov-report=xml

      - name: Upload coverage reports to Codecov
        if: matrix.python-version == '3.14'
        uses: codecov/codecov-action@v5
        with:
          token: ${{ secrets.CODECOV_TOKEN }}
          fail_ci_if_error: true
```

ПРИЛОЖЕНИЕ Б. ШПАРГАЛКА ПО КОМАНДАМ

Б.1 Команды GitHub CLI для работы с Actions

```
# список последних запусков
gh run list --workflow=ci.yaml --limit=10

# подробности запуска
gh run view <run-id>

# просмотр логов job'a
gh run view <run-id> --job=<job-id> --log

# ожидание завершения запуска
gh run watch <run-id>

# ручной запуск workflow (если настроен workflow_dispatch)
gh workflow run ci.yaml
```

Б.2 Локальные команды проекта

```
uv sync --frozen
uv run ruff check .
uv run ruff format . --check
uv run pyright
uv run pytest -m "unit or integration" -n auto
uv run pytest --cov=app --cov-fail-under=70 --cov-report=term-missing
uv run pre-commit run --all-files
docker compose up -d
docker compose logs -f app
docker compose down
```

Б.3 Минимальный шаблон workflow

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  check:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v5
      - uses: astral-sh/setup-uv@v7
        with:
          enable-cache: true
      - run: uv sync --frozen
      - run: uv run ruff check .
      - run: uv run ruff format . --check
      - run: uv run pyright
      - run: uv run pytest -m "unit or integration" -n auto
```